

Ejercicio 1. Familiarización con PVS

Nota: En estos ejercicios los pasos 1, 2, 3, 4, 12, 14, 22 y 23 serían para trabajar en línea, no en VSCODE

1. Inicia una sesión de PVS.
2. Crea una teoría y comenta su estructura. (`Alt-x nf`). `-- --.pvs`
3. Sal de PVS. (`C-x C-c`) o por ventana (`exit`)
4. Carga una teoría ya existente. (`Alt-x ff`)
5. Crea una función de $\mathbb{N} \mapsto \mathbb{N}$. Por ejemplo: $f(x) = 3x + 5$
6. Comprueba el tipado de la teoría. (`Alt-x tc`) (Debería dar un error)
7. Declara el tipo de la función creada. Ejemplo: $f(x) : nat = 3x + 5$
8. Comprueba de nuevo el tipado de la teoría.
9. Se necesita más información sobre los tipos a utilizar. Tenemos que declarar el tipo de la variable que se utiliza. En este caso el tipo de la variable x que se puede declarar antes o dentro de la propia función.
10. Crea una función de $\mathbb{N} \mapsto \mathbb{N}$. Por ejemplo: $g(x) = x - 2$. ¿Notas algún cambio en la teoría?
11. Formaliza un teorema basado en la función creada anteriormente. Comprueba de nuevo el tipado.

```
trivial: THEOREM (FORALL (x:nat): f(x) > x)
```

12. Para comenzar una prueba interactiva del teorema enunciado se tiene que situar el cursor sobre la declaración del teorema y escribir **Alt-x pr**
13. Prueba el teorema mediante el comando (**grind**)
14. Prueba de nuevo el teorema sin comandos "mágicos". Coloca de nuevo el cursor sobre la declaración del teorema y escribe en el buffer de pvs **Alt-x pr**
15. try again? (yes or no)
16. Rerun Existing proof? (yes or no)
17. Elimina el cuantificador universal (**skosimp**).
18. Vuelve atrás en la demostración con (**undo**)
19. Elimina el cuantificador universal (**skeep**). ¿Cuál es la diferencia con el anterior comando?
20. Expande la definición de la función f (**expand "f"**)
21. Aplica la estrategia (**assert**), que utiliza reescritura y realiza simplificaciones aritméticas para terminar la prueba.
20. Compara el Run Time y el Real Time de los dos métodos de prueba. Esto será importante cuando las fórmulas sean más complejas.
21. Would you like the proof to be saved? (Yes or No)
22. Teclea en el buffer **Alt-x spt**
23. Coloca el cursor encima de la palabra **THEOREM**, y teclea **Alt-x edit-proof** en el buffer.
24. Fin del ejercicio.
25. Si has acabado pronto añade a la teoría:

a: VAR nat

j(x): nat = 2*x + 2

otro: LEMMA j(2*a + 2 - j(a)) = 2

- Comprueba si está bien tipada.
- Mira el fichero *nombreTeoria.tccs*. Observa las obligaciones de prueba que aparecen, por ejemplo:

```
% Subtype TCC generated (at line 18, column 16) for a - f(a)
% expected type nat
% unfinished
otro_TCC1: OBLIGATION FORALL (a: nat): (2*a + 2 - f(a)) >= 0;
```

¿Por qué se ha generado esta obligación de prueba? ¿Se puede probar?

- Prueba el lema **otro**
- Comprueba el estado de prueba de la teoría y fíjate en lo que nos dice el sistema del teorema **trivial**
- Observa que la obligación de prueba **otro_TCC1** no está probada y el estatus del lema **otro** es **proved - incomplete**. Esto indica que este lema depende de otro resultado que no está probado. Tu prueba no es terminada hasta que el sistema ponga **proved - complete** en todos los teoremas y obligaciones de prueba.

26. Comprueba la validez o no de los secuentes:

$$\{(p \rightarrow q), (r \rightarrow s), q \wedge s \rightarrow t, \neg t\} \vdash \neg p \vee \neg r$$

$$\{(p \rightarrow q), (r \rightarrow s), q \wedge s \rightarrow t, \neg t\} \vdash \neg p \vee r$$

27. Prueba la validez del secuyente:

$$\{\forall x(P(x) \vee Q(x)), \exists x\neg Q(x), \forall x(R(x) \rightarrow \neg P(x))\} \vdash \exists x\neg R(x)$$